SOURCE-VERIFIED STUDY AND DEMONSTRATION GUIDE

GloVe: Global Vectors for Word Representation

Understand the original paper. Trace every training stage. Run the real Stanford C implementation. Explain the outputs to a professor.

PRIMARY PAPER
Jeffrey Pennington, Richard Socher, Christopher D. Manning — EMNLP 2014

REPOSITORY
github.com/stanfordnlp/GloVe — master branch, source inspected 11 November 2026

IMPLEMENTATION TARGET
Windows PC with Ubuntu/WSL2; original GCC-based C code, not a rewritten substitute

INCLUDED MATERIAL
Paper section walkthrough, mathematical derivation, real function trace, shell commands, live-demo plan, Q&A and pitfalls

IMPORTANT SCOPE The executable has not been run on your own computer from this chat. The included scripts are a tested-for-syntax starting point; present benchmark results only after running the Stanford code locally. The tiny smoke-test corpus is not a reproduction of the 2014 benchmark.

How to use this guide

Follow the material in this order: first explain what the model tries to encode; then derive the objective; then inspect the four process stages; then run the supplied local shell script; finally show the actual outputs, logs and one code-level update in your presentation.

Reading map

Part	Chapters	What you should be able to say
A — Paper	1–10	Why ratios? Why logarithms, bias, weighting and sparse updates? What did the experiments establish?
B — Code	11–18	What program runs next? What function calls what? Where are counts, gradients, vectors and outputs stored?
C — Run	19–23	Compile, execute, evaluate, debug and demonstrate on your own PC.
D — Defense	24–26	Answer professor questions, identify limitations and distinguish paper settings from demo settings.

Three statements to memorize

- The information: a word is characterized by global statistics of its neighboring words; informative patterns appear in ratios of conditional co-occurrence probabilities.
- The model: two learned vectors and biases approximate the log of an observed word-pair count, with a frequency-dependent weight.
- The code: the shell runs vocabulary construction → co-occurrence counting → record shuffling → threaded GloVe optimization → evaluation.

PART A · THE ORIGINAL PAPER

1. Abstract and the research question

The paper asks why word embeddings obtained from distributional data can exhibit interpretable linear relationships, such as a shared vector direction between two analogy pairs. Previous matrix methods leveraged aggregate corpus information; predictive window-based models were strong on analogies. GloVe was designed to combine global counts with a log-bilinear vector objective.

Global here refers to the statistics computed across the whole text corpus before fitting the vectors. It does not mean that the model uses a Transformer, or that every word pair is used in every update. The supplied implementation is C, not a PyTorch neural network.

Where GloVe fits

Approach	Training signal	Typical limitation / benefit
LSA / SVD	Term-document or word-context matrix, then factorization	Direct access to global statistics; transformations can have poor analogy geometry.
Skip-gram / CBOW	Prediction tasks over individual text windows	Strong useful vectors; updates revisit many repeated local windows.
GloVe	Observed global word-context counts	Compact regression targets; learn from distinct nonzero entries.

2. Section 2 — Related work

The paper reviews matrix-factorization methods (LSA, HAL, PPMI variants, HPCA) and window-based methods (skip-gram, CBOW, vLBL, ivLBL). Naively factorizing raw co-occurrence counts overemphasizes frequent function words, whereas window-based prediction can capture regularities but uses repeated examples. GloVe addresses both by transforming counts using logs and weighting the training error.

Do not say “word2vec ignores co-occurrence statistics.” It learns from them implicitly. The distinction is that GloVe explicitly constructs aggregate pair counts as its training targets.

3. Section 3 — The notation and the central insight

$$X_{ij} = \text{count}(j \text{ in context of } i), \quad X_i = \sum_k X_{ik}$$

$$P_{ij} = P(j \mid i) = \frac{X_{ij}}{X_i}$$

X_ij counts or accumulates how much word j appears in the context of word i. X_i is the total row mass; P_ij is the conditional co-occurrence probability. With distance weighting, X_ij can be a positive fractional value, not necessarily an integer.

The actual ice / steam example, Table 1

Context	P(k | ice)	P(k | steam)	Ratio
solid	1.9e-4	2.2e-5	8.9
gas	6.6e-5	7.8e-4	0.085
water	3.0e-3	2.2e-3	1.36
fashion	1.7e-5	1.8e-5	0.96

The ratio behaves like a filter: >1 favors ice, <1 favors steam, ≈1 is nondiscriminating. It does not mean fashion and water are equally common; it means their relative association to ice versus steam is similar. The model wants a difference of vectors to express such contrasts.

4. Section 3 — From ratios to dot products

The paper starts with an unknown function F of three word representations that should reproduce a ratio P_ik/P_jk. It then imposes requirements, not an arbitrary neural-network design.

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

Step 1: vector differences. The meaningful relation between target words i and j should depend on (w_i - w_j), so the model expresses a word relation as an offset in embedding space.

Step 2: scalar interaction. Taking the dot product (w_i - w_j)^T · context_k collapses the featurewise interaction to one number. The dot product can then be mapped to a positive ratio.

Step 3: exchange symmetry. A word can serve as a target or context. Switching target/context roles, along with transposing X, should not change the substance of the model. This motivates a consistent functional relationship and separate context vectors.

Step 4: exponential/logarithm. We need F(a-b)=F(a)/F(b). The exponential function satisfies this, because exp(a-b)=exp(a)/exp(b). Taking logs gives a linear additive relation.

$$w_i^T \tilde{w}_k = \log P_{ik} = \log X_{ik} - \log X_i$$

Step 5: biases. The row normalization log X_i does not depend on the specific context word k, so it can be absorbed into a target-word bias. An additional context bias restores target/context symmetry.

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$$

DEEP UNDERSTANDING The paper begins with probability ratios, but the executable does not directly calculate ratios in each gradient update. It fits log co-occurrence counts; the ratio structure arises when comparing the fitted equations for two target words. Explain this carefully.

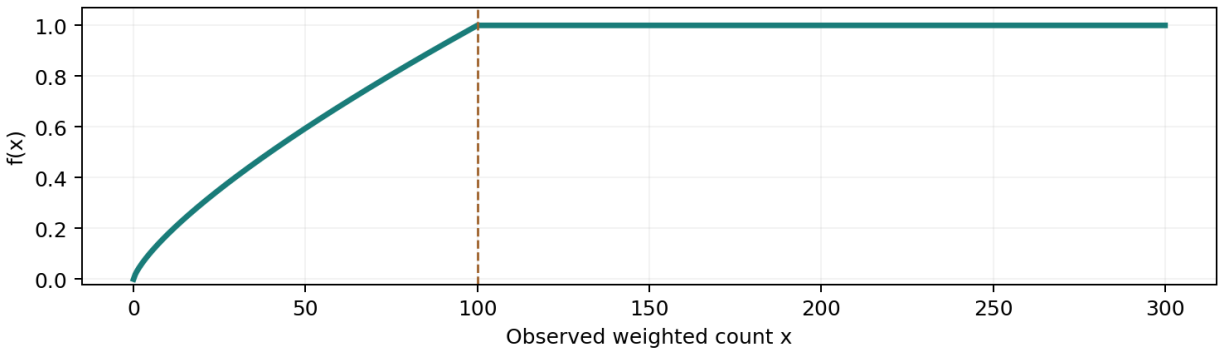
5. Section 3 — Why the weighted objective is necessary

If X_ij is zero, log X_ij is undefined. Replacing it by log(1+X_ij) is one possible factorization baseline discussed in the paper, not the core GloVe implementation. GloVe instead trains only on the nonzero observed entries of X, and introduces a weighting function.

$$J = \sum_{i,j\,:\,X_{ij}>0} f(X_{ij})\left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2$$

$$f(x) = \min\left(1, \left(\frac{x}{x_{max}}\right)^{\alpha}\right)$$

There are three key safeguards: f(0)=0, so zero pairs are ignored; rare co-occurrences receive small weights, because they are noisy; and very frequent co-occurrences saturate at weight 1 instead of growing without bound. The original experiments use x_max=100 and alpha=0.75.



Weighting curve for alpha=0.75 and x_max=100 (recreated from Eq. 9).

At x=1 and x_max=100, the weight is about 0.0316; at x=10 it is about 0.1778; for x≥100 it is 1. The weighting is a loss multiplier, not a softmax probability or a prediction target.

6. Derive a training update you can explain on a whiteboard

For an observed record (i,j,x), define prediction y = w_i dot context_j + b_i + context_bias_j, residual e = y - log x, and weight f=f(x). A standard per-record loss is L=f e². The theoretical gradients are:

$$\frac{\partial L}{\partial w_i} = 2fe\tilde{w}_j, \qquad \frac{\partial L}{\partial \tilde{w}_j} = 2few_i$$

$$\frac{\partial L}{\partial b_i} = 2fe, \qquad \frac{\partial L}{\partial \tilde{b}_j} = 2fe$$

In the source, the monitored cost is 0.5 × f × e², so the source uses f e rather than 2 f e in its update calculation. Multiplying a loss by 0.5 is a common convenience to remove the factor 2 in its gradient. Note that the exact AdaGrad accumulation and bias steps in this repository are implementation-specific; show the actual source, not generic pseudocode.

A numeric micro-example (illustrative, not trained output)

Suppose the count x=4, fitted dot product plus biases is 1.0, x_max=10 and alpha=0.75. Then log(4)≈1.386, so residual e≈−0.386. The weight is (4/10)^0.75≈0.503. The unscaled residual-weight product f×e is ≈−0.194, and the source-record monitored cost is ≈0.0375. An optimizer adjusts the vectors and biases to reduce that error.

7. Section 3.1 — Relationship to skip-gram and global objectives

The paper rewrites an online predictive objective by grouping repeated (word, context) events. Instead of a sum over many windows, the same objective can be expressed as a sum over distinct word pairs weighted by their counts: -sum_ij X_ij log Q_ij. This shows that window prediction also fundamentally operates on co-occurrence information.

From a softmax model Q_ij, the paper considers an unnormalized exponential model and a least-squares penalty on logarithms. Replacing the count-derived weight with f(X_ij) leads to an objective resembling GloVe. This is a conceptual derivation, not a claim that all versions of skip-gram and GloVe optimize identical losses.

8. Section 3.2 — Complexity and why sparsity matters

Let V be the vocabulary size, |C| the token count, d the embedding dimension, and nnz(X) the number of distinct observed word pairs. After count construction, one optimization pass requires roughly O(nnz(X) × d) arithmetic and stores two embedding matrices plus AdaGrad accumulators. The worst-case number of pairs is $O(V^2)$, but real word-context matrices are sparse.

The paper assumes a power-law model of pair-frequency ranks. It estimates an exponent around 1.25 and derives a scaling of nnz(X) ≈ O(|C|^0.8) under those assumptions. Do not confuse 1.25 here with the 0.75 exponent in the weighting function; they describe different things. The fitted power-law result is an empirical modeling argument, not a universal runtime guarantee.

9. Section 4 — Experiments, evaluation and conclusions

9.1 Experimental data and choices

The paper studies 1B-token Wikipedia 2010, 1.6B-token Wikipedia 2014, 4.3B-token Gigaword 5, 6B-token Wikipedia+Gigaword and 42B-token Common Crawl. It generally uses 400K vocabulary words (about 2M for Common Crawl), a symmetric window of ten positions on each side, inverse-distance co-occurrence contribution 1/d, x_max=100, alpha=.75, AdaGrad and an initial rate 0.05. Reported runs use 50 iterations for d<300 and 100 otherwise.

9.2 Evaluation tasks

- Word analogies: find the nearest vocabulary word to w_b - w_a + w_c, measured by cosine similarity, usually excluding the three query words; semantic and syntactic subsets are reported separately.
- Word similarity: compare cosine similarity rankings with human judgments (Spearman correlation).
- Named entity recognition: add word-vector features to a CRF-based system and report entity-level F1.

9.3 Selected original paper numbers (NOT local runs)

Configuration	Semantic	Syntactic	Overall analogies
GloVe 100d, 1.6B tokens	67.5%	54.3%	60.3%
GloVe 300d, 6B tokens	77.4%	67.0%	71.7%
GloVe 300d, 42B tokens	81.9%	69.3%	75.0%
SG (authors trained), 300d, 6B	73.0%	66.0%	69.1%

The headline 75.0% analogy result belongs to the 42B-token, 300d Common Crawl experiment. It is not an expected accuracy for the small text8 demo. The 6B result is 71.7%. An appropriate presentation explicitly labels every table as “paper result” or “my measured result.”

9.4 Sections 4.4–4.7: what the model analysis says

- Dimensionality: performance improves with dimension, then returns diminish (in the paper's measured settings).
- Window size: larger windows help semantic relations; smaller/asymmetric windows often help syntactic relations.
- Corpus: more text usually helps, but genre and coverage matter: Wikipedia supports geographical analogies well.
- Time comparisons: under a specific controlled setup, GloVe compares favorably with CBOW/skip-gram, but the comparison depends on tuning, data, hardware and training budget.

9.5 Section 5 and critical limitations

The authors conclude that count-based and prediction-based methods share a statistical foundation, and that explicitly fitting appropriately weighted global co-occurrences learns useful representations. A strong critical answer acknowledges static one-vector-per-word polysemy, rare/OOV words, corpus bias, preprocessing sensitivity, differences between benchmark setups, and the cost of building and storing counts. Unlike contextual language models, GloVe cannot assign a different embedding to "bank" in a river sentence versus a finance sentence.

PART B · THE ORIGINAL STANFORD C CODE

10. What is actually inside the repository?

Important root files: README.md (overview), demo.sh (orchestrator), Makefile (build), Training_README.md (later 2024 training documentation), src/common.[ch], src/vocab_count.c, src/cooccur.c, src/shuffle.c, src/glove.c, and eval/python/evaluate.py. The 2014 paper and current repository are related but not time-identical; the repo has later maintenance changes.

```
GloVe/
├── demo.sh                      # runs build, 4 stages, evaluation
├── Makefile                     # compiles 4 independent programs
├── src/
│   ├── common.h / common.c     # CREC, get_word, hash, CLI helpers
│   ├── vocab_count.c           # word frequency vocabulary
│   ├── cooccur.c               # weighted pair statistics
│   ├── shuffle.c               # randomize binary pair records
│   └── glove.c                 # training and vector export
└── eval/python/evaluate.py      # analogy test on exported vectors
```

CODE-CALL RULE These are distinct compiled programs. demo.sh starts processes in order and passes files between them. src/glove.c does not call get_cooccurrence(), shuffle_by_chunks() or get_counts(). Shared common.c functions are linked to all four executables by the Makefile.

11. The true execution order in demo.sh and Makefile

The default demo begins with make, downloads/unzips text8 if not present, declares parameters, executes the four binaries sequentially and runs the Python analogy evaluator when the corpus is text8. Redirection is significant: “< corpus” passes raw text to stdin, whereas “> output.bin” writes the program’s binary stdout to disk. Status messages go to stderr.

demo.sh literal setting	Value	Meaning
CORPUS	text8	A sample Wikipedia-derived text corpus; download step may need an HTTPS replacement.
VOCAB_MIN_COUNT	5	Discard words appearing fewer than five times.
WINDOW_SIZE / VECTOR_SIZE	15 / 50	15 context positions each side; 50-dimensional embeddings.
MAX_ITER / NUM_THREADS	15 / 8	15 full passes over shuffled counts; 8 worker threads.
X_MAX / MEMORY	10 / 4.0	Demo weighting cutoff 10; ~4 GB soft preprocessing memory limit.
BINARY	2	Export both vectors.txt and raw vectors.bin.

The Makefile builds object files and four executables, compiling each src/*.c with shared src/common.c and linking math and pthread libraries. There is no single giant train.py or neural-network layer stack in the original code.

12. Stage 1 — vocab_count.c

Exact calls and effects

main() [lines 149–170] reads -verbose, -max-vocab and -min-count and calls get_counts() [87]. The latter repeatedly uses get_word() from common.c to obtain whitespace-delimited tokens, calls hashinsert() [58] to increment counts in a hash table, transfers records into an array, sorts primarily by descending frequency (ties alphabetically), filters by min count, and prints “word count” lines into vocab.txt.

The vocabulary file order defines the 1-based integer IDs used later. These are not model embeddings yet; they are stable indices and frequency counts. The tokenizer is deliberately simple: it does not automatically perform sophisticated NLP preprocessing or remove punctuation.

```
build/vocab_count -min-count 5 -verbose 2 < text8 > vocab.txt
head -n 5 vocab.txt
wc -l vocab.txt
```

13. Stage 2 — cooccur.c

Build the sparse data for GloVe

main() [383–443] parses arguments, estimates the memory split between a dense region and an overflow buffer, then calls get_cooccurrence() [232]. The implementation loads vocab IDs into a hash table, scans the corpus, maintains a circular history buffer for up to window_size words, and creates pair records.

A preceding context word at distance d contributes 1/d when distance weighting is enabled. In symmetric mode the reverse ordered pair is added as well. Newline resets the history, so the C reader treats lines as window boundaries. Tokens missing from vocab are skipped. Therefore the counts are weighted, directed word-context pairs and need not be integer values.

For frequent rank combinations, counts accumulate in an in-memory table. Sparse overflow records are sorted into temporary files; merge_files() [164] uses a priority queue and merge_write() [153] to combine duplicate ordered pairs into the final binary stream. A C record is CREC {int word1; int word2; double val} as defined in common.h. Never open the .bin file in a text editor and expect plain words.

```
build/cooccur -memory 2.0 -window-size 15 -vocab-file vocab.txt \
  -symmetric 1 -distance-weighting 1 < text8 > cooccurrence.bin
```

One intuitive local window

For the toy phrase “small red apple”, with apple as the rightmost target and a left window of size two, red contributes 1 and small contributes 1/2 to the relevant ordered pair counts. With symmetric=1, their reverse ordered records are also accumulated. As the corpus is scanned, repeated pairs combine into one accumulated count.

14. Stage 3 — shuffle.c

main() [175–209] calls shuffle_by_chunks() [120], which reads binary CREC records into an array, calls shuffle() [57] using Fisher–Yates, writes temporary binary chunks, then calls shuffle_merge() [69] to combine chunks and shuffle across them. The source explicitly says the result is not necessarily a mathematically perfect global permutation.

Why shuffle? The co-occurrence stream from the previous stage is arranged by ordered word IDs; SGD on sorted pairs can have correlated successive updates. Shuffling makes the optimization order more random, while preserving the actual pair records and their values.

```
build/shuffle -memory 2.0 -seed 42 < cooccurrence.bin \
  > cooccurrence.shuf.bin
```

15. Stage 4 — glove.c: initialization to final output

The actual, source-confirmed call hierarchy

Caller and line	Callee / operation	Reason
main() [448]	find_arg(); count vocab lines; train_glove() [552]	Parse input and determine V.
train_glove() [379]	initialize_parameters() [396]	Allocate 2 × V × (d+1) parameters, plus accumulators.
initialize_parameters() [92]	load_init_file() [optional]; srand; gradsq=1	Random/start-from-checkpoint initialization.
train_glove() [415–425]	pthread_create(glove_thread); pthread_join	Each epoch assigns CREC ranges to workers.
glove_thread() [155–239]	Read CREC; compute error/weight; update W/gradsq	Actual optimization math per record.
train_glove() [432–445]	save_params() checkpoint/final	Write learned vectors to disk.
save_params() [242]	read vocab, write .txt and/or .bin	Map word IDs back to embedding rows.

What W actually stores

For each of V words, the program holds two separate parameter blocks: word vector plus bias and context vector plus bias. Each block occupies d+1 double values. Therefore total W elements = 2 × V × (d+1). A same-size gradsq array stores accumulated gradient-squared statistics, initially all ones. This is substantially different from a single V-by-d embedding table with no bias.

One source update, in the precise sequence

- Each worker opens shuffled binary input independently and seeks to the CREC range assigned by its thread ID [161–181].
- Read (word1, word2, val); compute word-vector offset l1 and context-vector offset l2, adjusting the 1-based word IDs [181–189].
- Calculate dot(w_i, context_j), add the two biases, subtract log(val) to obtain diff [190–193].
- Compute fdiff=f(val)×diff; add 0.5×fdiff×diff to the thread-local cost [194–202].
- Calculate per-dimension clipped update values, adaptive scaling with gradsq, and change both word/context parameters [204–224].
- Update the two bias terms and their squared-gradient state [226–231].
- End epoch: join threads, aggregate thread costs, print iteration cost, optionally checkpoint [423–440].
- After all epochs, call save_params(-1) [445] for final vectors.

RESEARCH-LEVEL IMPLEMENTATION NOTE The repository labels the optimizer adaptive-gradient / lock-free asynchronous SGD. The current source does not have a global lock around W updates, so multithreaded results can vary. Its precise update lines should be described as the implementation in glove.c, rather than assuming they match every textbook presentation of AdaGrad.

16. Output files and what each contains

Filename	Content	How to describe it
vocab.txt	word + unigram count	The vocabulary and ordering of word IDs.
cooccurrence.bin	ordered CREC records	Accumulated, weighted word-context counts.
cooccurrence.shuf.bin	same CREC entries randomized	The actual training-input dataset.
vectors.txt	word followed by d floats, plus optional <unk>	Final human-readable embeddings: default model=2 sums word+context vectors.
vectors.bin	2 × V × (d+1) doubles	Raw word/context parameters including biases; NOT the same format as vectors.txt.
logs/train.log	epoch costs and settings (from our script)	Evidence of a genuine local training run.

The default -model 2 for text export returns w_i + context_i rather than one of the two vectors alone. The raw binary export saves both original parameter blocks. -binary 2 writes both output formats. The code also constructs a default <unk> row from the least-frequent up-to-100 vocabulary words for text output.

Subtle source detail: the help text describes a -model 3 concatenation option, but the current main() branch coerces every value other than 0 or 1 to 2 [lines 522–523]. Thus do not demonstrate -model 3 as reliable in this source revision. This is a good example of reading the implementation instead of blindly trusting help text.

17. Code flow diagram and where to point during your demo

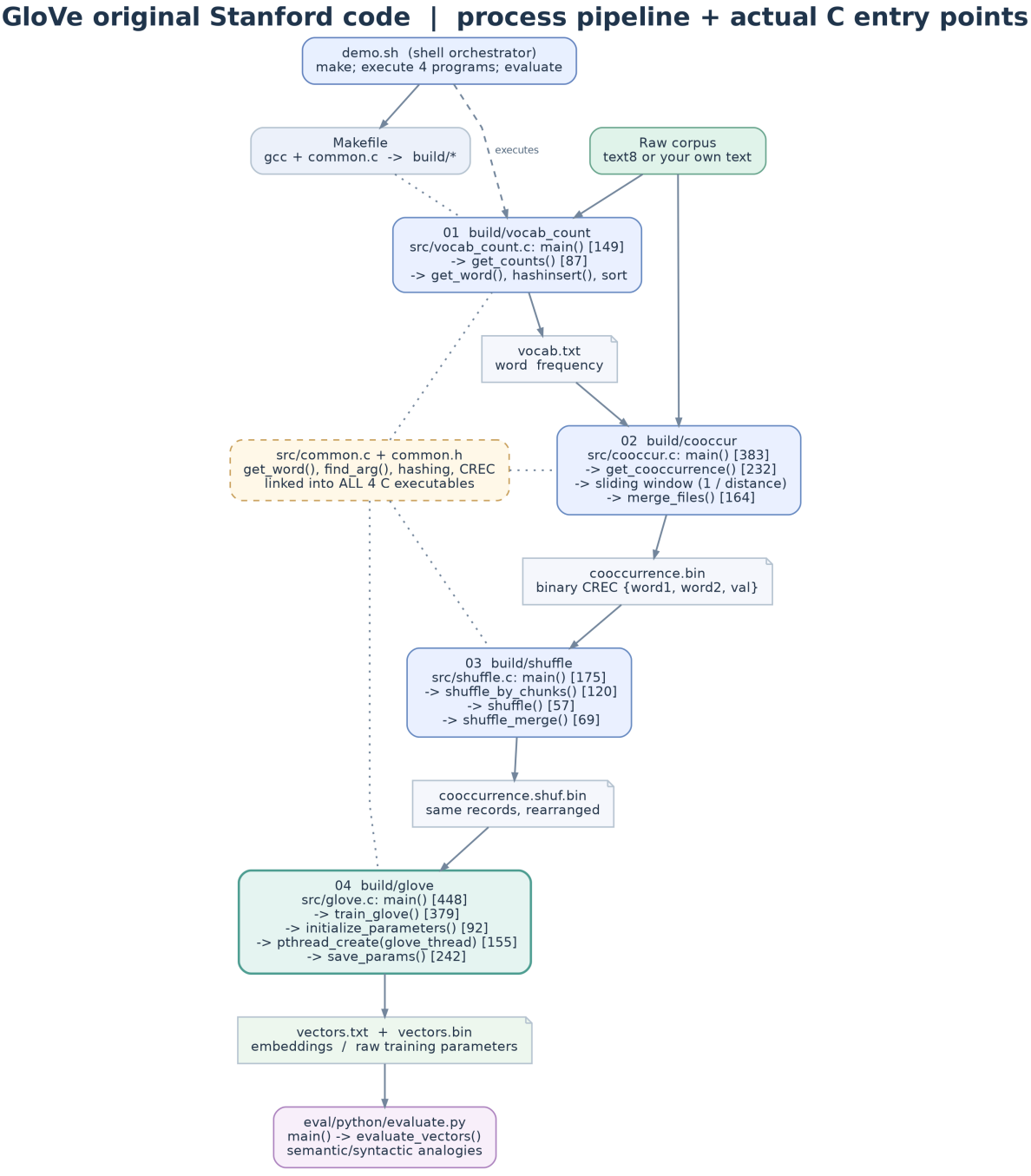


Figure A. Overall process boundaries, file artifacts, and actual source entry points. A high-resolution diagram is supplied separately.

Use Figure A for the first code slide. Then switch to the separate deep training call tree while navigating src/glove.c. A third diagram maps helper functions inside preprocessing and evaluation. All source line numbers refer to the master branch inspected for this guide; line numbers may move after future commits.

18. eval/python/evaluate.py — what the score means

The official Python script loads vocab.txt and vectors.txt, constructs a matrix in vocabulary order, normalizes each word vector, and evaluates pre-provided analogy questions. For A:B::C:?, the code uses B - A + C and dot product with the normalized matrix to rank candidate words, excluding the three query words. It prints coverage, semantic accuracy, syntactic accuracy and overall top-1 accuracy.

Two cautions: coverage counts only questions whose words are in the learned vocabulary; and tiny toy-data vectors are unsuitable for the official analogy evaluation. The original script assumes that the evaluation data directory exists and its categories have usable questions; do not confuse missing-vocabulary errors with a training failure.

PART C · IMPLEMENT LOCALLY ON YOUR WINDOWS PC

19. Recommended setup: Ubuntu under WSL2

The original project is built with GCC, POSIX functions, pthreads, Make and shell redirection. Its easiest path on Windows is WSL2 Ubuntu, not trying to compile it as a native Visual Studio project. These commands run in Windows PowerShell (administrator) and then the Ubuntu shell.

```
## Windows PowerShell (administrator), if WSL is not installed
wsl --install -d Ubuntu
## Restart if prompted; open Ubuntu and create your Linux account.

## Ubuntu shell
sudo apt update
sudo apt install -y build-essential git make wget unzip python3 python3-venv
python3 -m venv ~/venvs/glove-demo
source ~/venvs/glove-demo/bin/activate
python3 -m pip install --upgrade pip numpy
```

If WSL was already installed, wsl --status from PowerShell tells you more about the environment. If your Windows environment forbids WSL, a Linux VM or compatible native Linux machine is a straightforward alternative.

20. Clone and compile the original repository

```
cd ~
git clone https://github.com/stanfordnlp/GloVe.git
cd GloVe
git rev-parse --short HEAD
make
ls -l build/vocab_count build/cooccur build/shuffle build/glove
build/glove                 # shows usage when no args
```

Store the git commit hash that you actually checked out. It gives you an exact version to cite and matches your source-level explanation. You should see four compiled executable binaries. If

gcc/make fails, install build-essential; if a large project runs out of memory, reduce preprocessing -memory.

21. Two run modes: smoke test first, real demonstration second

Mode A: tiny smoke test (tests the pipeline, not semantic quality)

Copy the supplied run_glove_local.sh and probe_vectors.py into the cloned repository root. If you saved them in the Windows Downloads folder, a simple option from Ubuntu is:

```
# Example only: change <your-Windows-user> to your real Windows login
cp /mnt/c/Users/<your-Windows-user>/Downloads/run_glove_local.sh .
cp /mnt/c/Users/<your-Windows-user>/Downloads/probe_vectors.py .
bash run_glove_local.sh tiny
python3 probe_vectors.py vectors.txt king queen paris london
```

This creates a tiny synthetic corpus with repeated examples, compiles the original source, writes all intermediate files, trains 25-dimensional embeddings and saves logs. Expect to inspect file presence and epoch costs; do not expect reliable semantic analogies from such small artificial training data.

Mode B: text8 — practical train-and-evaluate demo

The repository demo.sh uses text8, a ~100MB text corpus and a 50-dimensional model. For a more meaningful experiment, fetch text8 using HTTPS (the original demo script contains an older HTTP URL that may be blocked by some networks).

```
cd ~/GloVe
wget -O text8.zip https://mattmahoney.net/dc/text8.zip
unzip -o text8.zip
ls -lh text8
bash run_glove_local.sh text8
python3 eval/python/evaluate.py \
  --vocab_file vocab.txt --vectors_file vectors.txt
python3 probe_vectors.py vectors.txt king queen man woman
```

The included script explicitly sets the demo parameters and seed values, but with 8 lock-free threads you may still observe small numerical differences across runs. The text8 corpus download and execution must happen on your PC; no benchmark scores have been claimed in advance. If text8 hosting changes, obtain it from a trustworthy mirror or use an appropriately preprocessed own corpus.

Alternative: run Stanford’s original unmodified demo

```
cd ~/GloVe
bash demo.sh
```

The original demo itself executes make → vocab_count → cooccur → shuffle → glove → evaluate.py. If its HTTP download fails, manually download/unzip text8 beforehand as shown above; then rerun without changing Stanford’s source. The demo sets its own parameters independently from the paper’s larger experiments.

22. How to prove to your professor you actually ran the code

Suggested live evidence in execution order

- Show git rev-parse --short HEAD, repo root and successful make. Explain why there are four compiled programs.
- Show the first lines and line count of vocab.txt. Explain word count, filtering and 1-based indexing.
- Show ls -lh cooccurrence.bin cooccurrence.shuf.bin. Explain CREC and why these are binary, not text.
- Show the actual build/glove command and your logs/train.log from the run. Point to decreasing/variable objective values; never promise strict monotonicity.
- Show vectors.txt (one token plus d numeric columns; optional <unk>). Explain word+context export.
- Run the official analogy evaluator on text8, then the supplied probe. State both the score and in-vocabulary question coverage.
- Open src/glove.c at main() [448], train_glove() [379] and glove_thread() [155] and match the deep call tree.

FOR YOUR PROFESSOR Take screenshots only after your local run: build output, vocabulary file, actual training cost lines, vectors.txt and evaluation. Describe anything you did not measure as a demonstration plan, not a completed result.

23. Troubleshooting and common command mistakes

Problem	Most likely fix
make: command not found / gcc missing	sudo apt install build-essential
./demo.sh: Permission denied	bash demo.sh (avoids executable-bit dependence)
text8 cannot be downloaded	Use the HTTPS wget command, or download manually and unzip.
No vectors.txt	Check logs/train.log; -binary 1 saves only binary; use -binary 2 or 0.
Large RAM / disk usage	Use -memory 0.25 or 2.0; reduce corpus for smoke test; clean temporary artifacts.
A word not found	It may be below -min-count, absent from text8, or preprocessed differently.
Analogy results differ from paper	text8 is smaller; model settings, window, vocab, iterations and seed differ.
Output differs across repeated runs	Lock-free multithreading and randomized data order can change optimization.
Python NumPy error	Activate your venv and run python3 -m pip install numpy.

PART D · QUESTIONS, CRITIQUE AND TALKING POINTS

24. Fast oral questions and answers

Q1. What is GloVe in one sentence?

A global count-based log-bilinear word embedding model that fits weighted logarithms of observed word-context co-occurrences.

Q2. Why ratios rather than raw probabilities?

Ratios cancel context words that associate equally with two targets and highlight distinguishing attributes.

Q3. Why does the objective use log(X_ij)?

Log converts multiplicative ratios to additive differences and compresses the scale of counts.

Q4. Does the code compute ratios each step?

No. It fits log count records; the ratio interpretation motivates the model mathematically.

Q5. Why target and context vectors?

The training association is between target i and context j; separate roles and biases preserve flexibility and symmetry.

Q6. Why biases?

They absorb marginal-frequency effects and restore the desired target/context exchange symmetry.

Q7. Why f(x)?

To suppress unreliable rare co-occurrences and prevent frequent pairs from growing unbounded in importance.

Q8. What are alpha and x_max?

The exponent and saturation cutoff for f(x). The paper uses .75 and 100; demo.sh uses .75 and x_max=10.

Q9. Why are there zero-free training records?

The model skips zero pairs instead of explicitly computing log(0); it operates on observed nonzero entries.

Q10. Is GloVe an RNN / CNN / Transformer?

No. This implementation optimizes vector and bias lookup parameters with a weighted regression loss.

Q11. What is CREC?

A packed binary record type in common.h holding word1 and word2 integer IDs and a double-valued weighted count.

Q12. What does cooccur do that glove does not?

It scans raw corpus windows and constructs global pair counts; glove consumes those aggregated binary counts.

Q13. Why shuffle?

To randomize the order of training records and avoid a deterministic frequency-ID order during stochastic updates.

Q14. Why two passes over the text corpus?

vocab_count learns the vocabulary; cooccur then uses its IDs to collect weighted co-occurrences. Training later uses the resulting matrix records.

Q15. How is training parallelized?

train_glove creates pthread workers that read disjoint record segments while updating shared W and gradsq without global locks.

Q16. What optimizer?

The authors describe AdaGrad; the code performs adaptive per-parameter updates with squared-gradient accumulators and clipping.

Q17. What is the exact objective?

Sum over observed pairs of f(X_ij) times the squared difference between fitted dot product plus biases and log(X_ij).

Q18. Why does train.log cost contain a factor 0.5?

The source accumulates $0.5*f*error^2$, which removes factor 2 from per-example derivatives.

Q19. What is in vectors.txt?

With default model=2, one token followed by the sum of its target and context embedding vectors.

Q20. How is vectors.bin different?

It is a raw double-precision dump of two parameter blocks per word, including biases, rather than a text embedding list.

Q21. Is 75% your measured local score?

No: it is the original paper's 42B-token 300d analogy result. I report my local score separately after running text8.

Q22. Are the results deterministic?

Not strictly with lock-free multithreading, even with fixed random seeds.

Q23. Why can't the toy run pass analogy benchmarks?

It has too little varied language data, very limited vocabulary and artificial repeated contexts.

Q24. What is GloVe's main weakness?

Static vectors do not disambiguate senses by sentence context; rare words, preprocessing and corpus bias also affect results.

Q25. What is the cost of one epoch?

Approximately O(nnz(X) × dimension), after the one-time expensive count-construction stage.

Q26. What is the relation between the code and paper?

The C implementation embodies the same weighted log-count model, but demo settings and later repository maintenance differ from 2014 experimental setups.

25. A credible 15-minute talk sequence — once you are ready

Your requested task allows five minutes on theory, ten on code and five for Q&A.; For learning, take as long as you need; once prepared, this is a succinct transition plan you can use.

Segment	Show / explain
Paper opening (0–2)	Problem and ratio example (ice / steam); contrast with SVD and skip-gram.
Paper derivation (2–4)	Co-occurrence probability, log equation, biases, weighting f, key formula.
Paper results (4–5)	Original benchmark numbers with corpus/dim labels; limitations.
Code architecture (5–7)	demo.sh and full pipeline diagram; distinct processes, stdin/stdout.
Preprocessing (7–10)	vocab_count.c, cooccur.c, shuffle.c; outputs and 1/d weighting.
Actual training (10–14)	main → train_glove → initialize_parameters → glove_thread → save_params; walk through a CREC gradient update.
Real demo (14–15)	Your compile, logs, vectors, evaluator and one nearest-neighbor example.

26. Professor readiness: final checklist

- ☐ I can explain the ice/steam ratios without reading a slide.
- ☐ I can derive the log-count equation from a ratio and explain both biases.
- ☐ I can distinguish log(1+X) SVD baseline from GloVe's nonzero log(X) regression.

- ☐ I can explain why X is sparse and why count construction is a separate phase.
- ☐ I can trace each program and its actual file outputs.
- ☐ I can open src/glove.c and identify main(), train_glove(), initialize_parameters(), glove_thread(), save_params().
- ☐ I can explain where word/context IDs, biases, fdiff and gradsq live.
- ☐ I have independently run make, the tiny smoke test, and the text8 evaluation when feasible.
- ☐ I have saved the exact commit hash and screenshots or logs of my run.
- ☐ I label the published 75.0% separately from my own measured scores.
- ☐ I can explain one bug/quirk or implementation caveat with supporting source lines.

Sources and direct source-code URLs

Original research paper: https://aclanthology.org/D14-1162.pdf

Repository: https://github.com/stanfordnlp/GloVe

Stanford project page: https://nlp.stanford.edu/projects/glove/

Orchestration: https://github.com/stanfordnlp/GloVe/blob/master/demo.sh

Build: https://github.com/stanfordnlp/GloVe/blob/master/Makefile

Tokenizer / data structures: https://github.com/stanfordnlp/GloVe/blob/master/src/common.h

Vocabulary: https://github.com/stanfordnlp/GloVe/blob/master/src/vocab_count.c

Co-occurrences: https://github.com/stanfordnlp/GloVe/blob/master/src/cooccur.c

Shuffle: https://github.com/stanfordnlp/GloVe/blob/master/src/shuffle.c

Trainer: https://github.com/stanfordnlp/GloVe/blob/master/src/glove.c

Evaluation: https://github.com/stanfordnlp/GloVe/blob/master/eval/python/evaluate.py

Licensing: The Stanford repository is licensed under Apache 2.0. This guide uses its structure and function names to explain the original source; it is not a replacement for citing the authors or running the official implementation.